COMPUTER ENGINEERING

QUEZON CITY

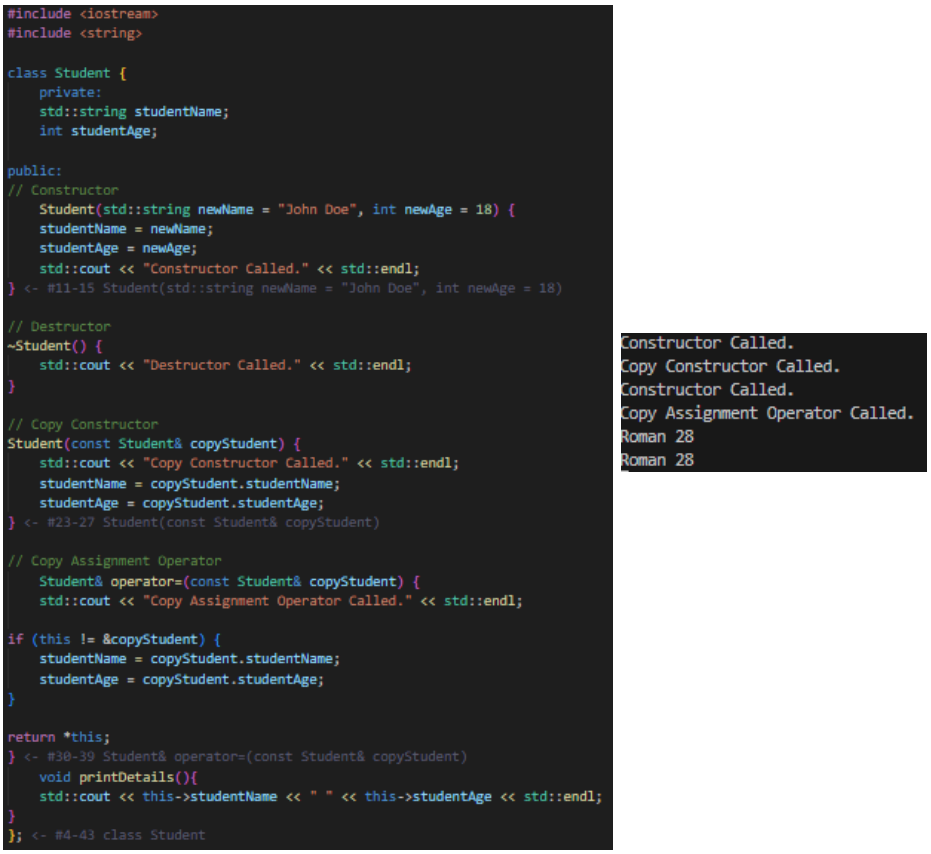
COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	07-17-2026	**DATE SUBMITTED:**	07-22-2026
NAME:	GUARIN, JAN MIKAELA C.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 2.1 Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 2-1. Initial Driver Program

Screenshot	
Observation	The program successfully demonstrated the use of the constructor, copy constructor, and copy assignment operator. The constructor initialized the first and third Student objects, the copy constructor created a copy of the first object into the second object, and the copy assignment operator copied the data from the second object to the third object.

```
#include <iostream>
#include <string>

class Student {
    private:
    std::string studentName;
    int studentAge;

public:
// Constructor
    Student(std::string newName = "John Doe", int newAge = 18) {
    studentName = newName;
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
} <- #11-15 Student(std::string newName = "John Doe", int newAge = 18)

// Destructor
~Student() {
    std::cout << "Destructor Called." << std::endl;
}

// Copy Constructor
Student(const Student& copyStudent) {
    std::cout << "Copy Constructor Called." << std::endl;
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
} <- #23-27 Student(const Student& copyStudent)

// Copy Assignment Operator
    Student& operator=(const Student& copyStudent) {
    std::cout << "Copy Assignment Operator Called." << std::endl;

if (this != &copyStudent) {
    studentName = copyStudent.studentName;
    studentAge = copyStudent.studentAge;
}

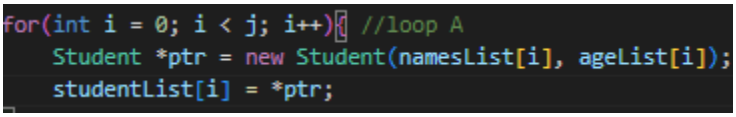
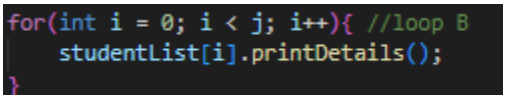
return *this;
} <- #30-39 Student& operator=(const Student& copyStudent)
    void printDetails(){
    std::cout << this->studentName << " " << this->studentAge << std::endl;
}
}; <- #4-43 class Student
```

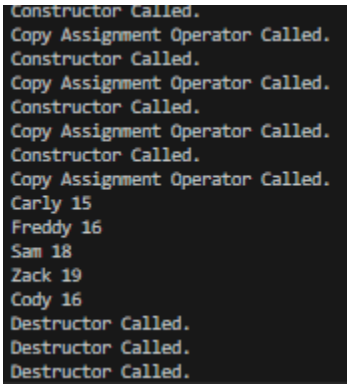
```
Constructor Called.
Copy Constructor Called.
Constructor Called.
Copy Assignment Operator Called.
Roman 28
Roman 28
```

Table 2-2. Modified Driver Program with Student Lists

Screenshot	
Observation	After modifying, the program uses three arrays which are the nameList to store the student names, ageList to store the student ages, and studentList to store student objects. The arrays are modified to 5 which demonstrates static object allocation. Each student's information is stored in their corresponding indexes.

Table 2-3. Final Driver Program

Loop A	
Observation	This program creates a new student object for each iteration using “new”. It initializes the object with their corresponding name and age which came from the nameList and ageList arrays, then it copies it into the studentList array.
Loop B	
Observation	Loop B iterates through the studentList array and calls the printDetails to display every name and age of students which were created in Loop A.

Output	
Observation	The program successfully displayed the details of all five students including their name and age. The output also displayed messages from constructors whenever a student's object was copied. This confirms that the program uses the copy constructor to duplicate the objects before storing or processing them.
Modifications	`delete ptr;`
Observation	The program was modified by adding delete ptr; after copying each dynamically allocated Student object into the studentList array. This ensures that the memory allocated with the new operator is properly released after it is no longer needed. The modification prevents memory leaks and demonstrates proper memory management when using dynamic allocation in C++.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>
#include <string>
using namespace std;

class Fruit
{

private:
    string name;
    int price;
    int quantity;

public:
    Fruit(string n = "None", int p = 0, int q = 0)
    {
        name = n;
        price = p;
        quantity = q;
    }

    Fruit(const Fruit &f)
    {
        name = f.name;
        price = f.price;
        quantity = f.quantity;
    }

    Fruit &operator=(const Fruit &f)
    {
        if (this != &f)
        {
            name = f.name;
            price = f.price;
            quantity = f.quantity;
        }

        return *this;
    }

    ~Fruit()
    {
    }

    int calculateSum()
    {
        return price * quantity;
    }

    void display()
    {
        cout << "Name: " << name << endl;
        cout << "Price: " << price << endl;
        cout << "Quantity: " << quantity << endl;
        cout << "Subtotal: " << calculateSum() << endl;
        cout << endl;
    }
};
```

```cpp
class Vegetable
{

private:
    string name;
    int price;
    int quantity;

public:
    Vegetable(string n = "None", int p = 0, int q = 0)
    {
        name = n;
        price = p;
        quantity = q;
    }

    Vegetable(const Vegetable &v)
    {
        name = v.name;
        price = v.price;
        quantity = v.quantity;
    }

    Vegetable &operator=(const Vegetable &v)
    {

        if (this != &v)
        {
            name = v.name;
            price = v.price;
            quantity = v.quantity;
        }

        return *this;
    }

    ~Vegetable()
    {

        if (name == "Lettuce")
        {
            cout << "Destructor called for " << name << endl;
        }
    }

    int calculateSum()
    {
        return price * quantity;
    }

    void display()
    {

        cout << "Name: " << name << endl;
        cout << "Price: " << price << endl;
        cout << "Quantity: " << quantity << endl;
        cout << "Subtotal: " << calculateSum() << endl;
        cout << endl;
```

Figure 1. Code Snippet of Fruit and Vegetable Class Implementation

In Figure 1, I created the Fruit and Vegetable classes to organize the grocery items based on their category. I declared the attributes name, price, and quantity to store the information of each item. I used constructors to initialize the values of the objects when they are created. I also included the copy constructor and copy assignment operator to allow objects to be copied and assigned properly. The destructor is used to handle the removal of objects, while the calculateSum() and display() functions are used to calculate and show the details of each grocery item.

```
int TotalSum(Fruit *fruitList[], int fruitSize,
             Vegetable *vegetableList[], int vegetableSi
{

    int total = 0;

    for (int i = 0; i < fruitSize; i++)
    {
        total += fruitList[i]->calculateSum();
    }

    for (int i = 0; i < vegetableSize; i++)
    {
        total += vegetableList[i]->calculateSum();
    }

    return total;
}
```

Figure 2. Code Snippet of TotalSum() Function

In Figure 2, I created the TotalSum() function to calculate the total amount Jenna needs to pay. The function receives the arrays containing the Fruit and Vegetable objects and uses loops to access every item. Each item's subtotal is added together by calling the calculateSum() function, and the final value is returned as the overall grocery cost.

```
int main()
{

    Fruit *GroceryListFruit[2];

    Vegetable *GroceryListVegetable[2];

    GroceryListFruit[0] = new Fruit("Apple", 10, 7);
    GroceryListFruit[1] = new Fruit("Banana", 10, 8);

    GroceryListVegetable[0] = new Vegetable("Broccoli", 60, 12);
    GroceryListVegetable[1] = new Vegetable("Lettuce", 50, 10);

    for (int i = 0; i < 2; i++)
    {
        GroceryListFruit[i]->display();
    }

    for (int i = 0; i < 2; i++)
    {
        GroceryListVegetable[i]->display();
    }

    cout << "Total: "
         << TotalSum(GroceryListFruit, 2,
                     GroceryListVegetable, 2)
         << endl;

    cout << endl;

    Fruit copiedFruit = *GroceryListFruit[0];

    Vegetable copiedVegetable = *GroceryListVegetable[0];

    Fruit assignedFruit;

    assignedFruit = *GroceryListFruit[1];
```

Figure 3. Code Snippet of main() Function

In Figure 3, I used the main() function as the driver of the program. In this part, I created the grocery list arrays and used dynamic memory allocation through the new operator to create objects for Apple, Banana, Broccoli, and Lettuce. I displayed the details of each item, calculated the total payment, and tested the copy constructor and copy assignment operator.

```
delete GroceryListVegetable[1];

GroceryListVegetable[1] = nullptr;

cout << "Lettuce has been deleted." << endl;

return 0;
```

Figure 4. Code Snippet of Lettuce Deletion and Memory Deallocation

In Figure 4, I used the main() function as the driver of the program. In Figure 4, I used the delete operator to remove the Lettuce object from the grocery list. Since the object was created using dynamic memory allocation, the allocated memory must be released after use. I assigned the pointer to nullptr after deletion to ensure that it no longer refers to the removed object.

Summary of Lessons Learned

In this activity, I learned how to apply Object-Oriented Programming (OOP) concepts in creating a grocery list system. I learned how to create classes, objects, constructors, destructors, copy constructors, and copy assignment operators. I also learned how to use functions to process data, such as calculating the total price of grocery items. Additionally, I gained a better understanding of dynamic memory allocation using new and delete, as well as the importance of properly managing memory when creating and removing objects.

Concluding Statement / Feedback

I think I performed well in this activity because I was able to apply the concepts discussed in class and create a working program that follows the given requirements. This activity helped me improve my understanding of classes, objects, and memory management in C++. However, I still need to improve my skills in writing more efficient code and understanding more advanced OOP concepts. I also need more practice in debugging and analyzing errors to become more confident in programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: Code assistance and explanation of programming concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: AI was used as a guide in understanding C++ concepts and organizing the code structure. The ideas, analysis, and final decisions regarding the program were reviewed and understood by me. AI assistance did not replace my own learning, problem-solving, or completion of the required activity.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.